

Analiza wydajności modeli YOLOv5 w detekcji obiektów kampusowych

Aleksander Święch s29379

1. Zbiór danych

1.1 Zbieranie danych

Zbiór danych został zebrany na kampusie PJATK przy użyciu aparatu Samsung Galaxy Note 20 Ultra. Skupiłem się na czterech klasach obiektów powszechnie występujących na terenie uczelni:

- **Czerwony leżak** - charakterystyczne siedziska rekreacyjne
- **Czarny leżak** - podobne do czerwonych, ale w odmiennej kolorystyce
- **Betonowa ławka** - standardowe ławki parkowe
- **Szary śmietnik** - kosze na śmieci o podobnym kształcie do ławek

1.2 Charakterystyka zbioru danych

Oryginalny zbiór:

- 340 zdjęć w rozdzielczości 12MP
- Pełnoklatkowe ujęcia z aparatu Samsung Galaxy Note 20 Ultra
- Etykietowanie wykonane w narzędziu makesense.ai

Rozkład klas w finalnym zbiorze (z augmentacją):

- **Czarny leżak** (black_lounger): 349 obiektów (17.7%)
- **Czerwony leżak** (red_lounger): 458 obiektów (23.2%)
- **Betonowa ławka** (concrete_bench): 793 obiekty (40.2%)
- **Szary śmietnik** (gray_bin): 374 obiekty (18.9%)
- **Łącznie**: 1974 oznaczone obiekty

Ławki stanowią dominującą klasę ze względu na ich powszechność na kampusie i łatwość oznaczania w różnych ujęciach.

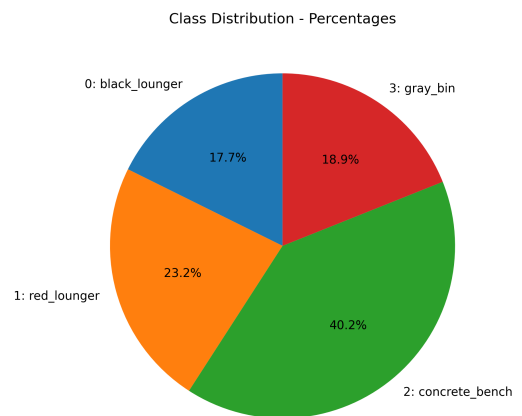
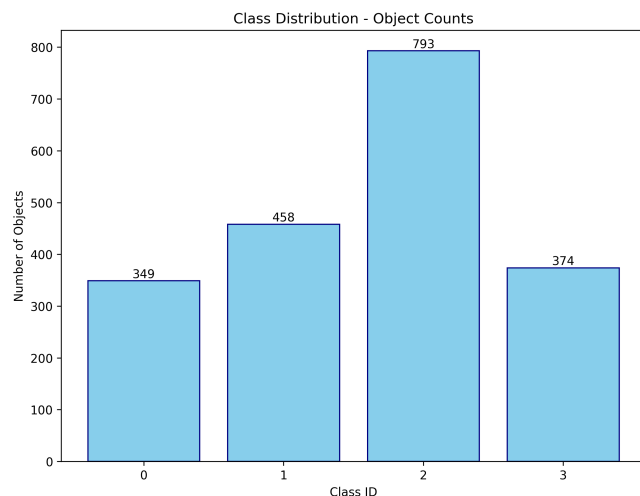
Problem wykryty podczas pierwszych testów: Filmy testowe były nagrywane tym samym urządzeniem, ale z funkcją sensor crop w rozdzielczości 1080p. Spowodowało to znaczną różnicę w skali i perspektywie obiektów między danymi treningowymi a testowymi.

1.3 Augmentacja danych

Po słabych wynikach pierwszych modeli dodałem **160 klatek z filmów testowych** do zbioru treningowego, aby:

- Poprawić detekcję mniejszych obiektów (efekt sensor crop)
- Dodać różnorodne perspektywy i kąty widzenia
- Uwzględnić motion blur i inne artefakty charakterystyczne dla nagrań wideo

Taka augmentacja okazała się konieczna - bez niej modele radziły sobie tragicznie mimo idealnych metryk treningowych.



2. Metodologia

2.1 Modele i konfiguracja

Przetestowałem 3 rozmiary YOLOv5: nano (~1.9M parametrów), small (~7.2M) i medium (~21M). Każdy trenowałem w dwóch wariantach: pretrained (z wagami COCO) i from scratch (losowe wagi).

Parametry treningu: workers=8, batch_size=8, img_size=640px, epochs=50, device=CPU (AMD Ryzen 9 7900X, 32GB RAM)

Czasy treningu: YOLOv5n (~20 min), YOLOv5s (~1-1.5h), YOLOv5m (~3-4h). Wszystkie 6 modeli trenowały się sekwencyjnie przez noc i poranek (~12h łącznie).

3. Wyniki i analiza

3.1 Pierwotne problemy (bez augmentacji)

Modele trenowane wyłącznie na pełnoklatkowych zdjęciach wykazywały poważne problemy podczas testów na filmach. Wersje pretrained potrafiły wykrywać niektóre obiekty, ale klasyfikacja była nieprecyzyjna, a większość obiektów pozostawała niewykryta ze względu na różnice w skali między danymi treningowymi a testowymi.

Modele trenowane from scratch prezentowały jeszcze gorsze wyniki. Generowały liczne false positives, wykrywając obiekty tam gdzie ich nie było, a jednocześnie praktycznie nie dokonywały poprawnej klasyfikacji rzeczywistych obiektów. Paradoksalnie, metryki treningowe wyglądały bardzo dobrze, co świadczyło o overfittingu do zbioru treningowego.

3.2 Wyzwania klasyfikacyjne

Wybrane klasy stwarzały dodatkowe wyzwania:

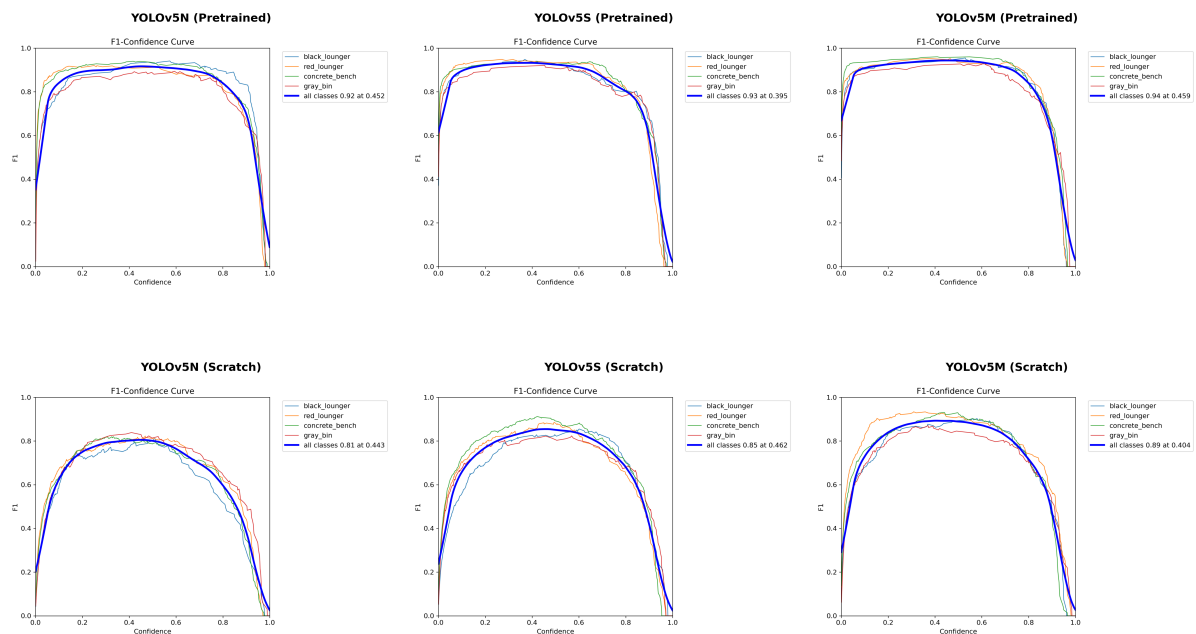
- Leżaki są do siebie bardzo podobne (różni je głównie kolor)
- Ławka i śmietnik mają podobny prostokątny kształt
- Test sprawdzał, czy model potrafi rozróżnić subtelne różnice między klasami

3.3 Analiza po augmentacji

Po dodaniu 160 klatek z filmów do zbioru treningowego sytuacja znacznie się poprawiła, ale pojawiły się nowe problemy specyficzne dla każdego typu modelu.

4. Porównanie modeli

Model Comparison: F1 Curve



4.1 Metryki walidacyjne - analiza szczegółowa

Modele pretrained - wysokie wyniki:

- YOLOv5s_pretrained: najwyższy $mAP@0.5 = 0.967$ (96.7%)
- YOLOv5m_pretrained: $mAP@0.5 = 0.971$ (97.1%)
- YOLOv5n_pretrained: $mAP@0.5 = 0.964$ (96.4%)

Modele from scratch - znacznie słabsze:

- YOLOv5s_scratch: $mAP@0.5 = 0.912$ (91.2%)
- YOLOv5m_scratch: $mAP@0.5 = 0.937$ (93.7%)
- YOLOv5n_scratch: $mAP@0.5 = 0.857$ (85.7%)

Analiza confusion matrix: Wszystkie modele pretrained wykazują podobne wzorce błędów:

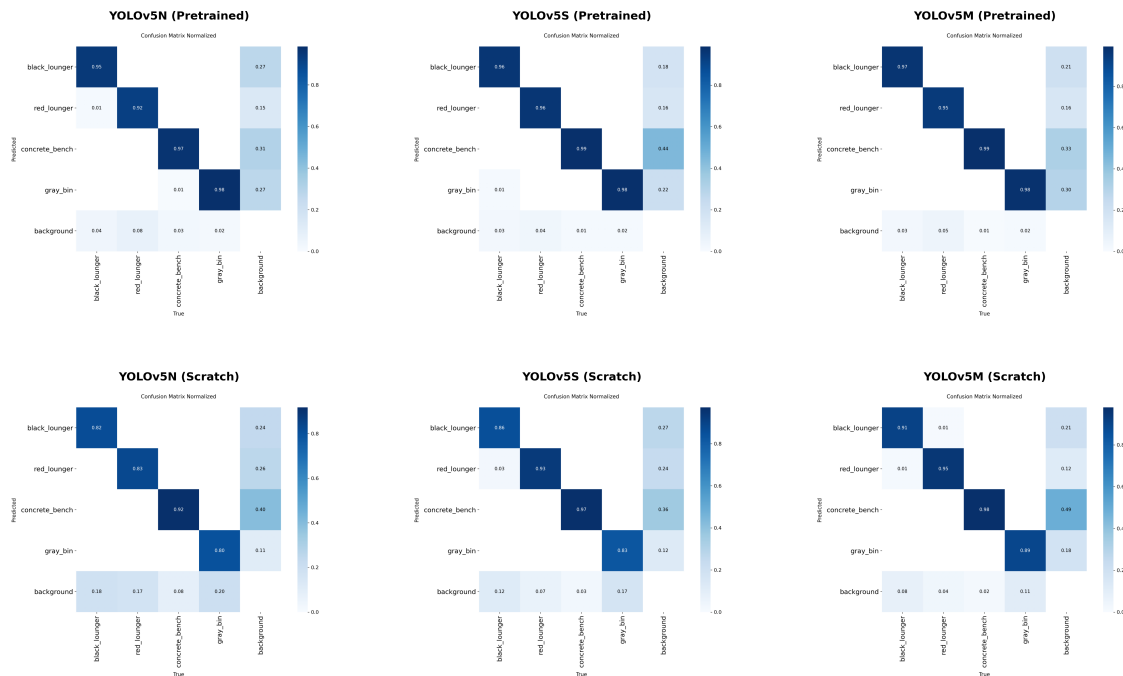
- Łóżaki czerwone vs czarne: najczęstsza konfuzja, modele mylą kolory
- Ławka vs śmietnik: drugi najczęstszy błąd ze względu na podobny kształt prostokątny
- Betonowa ławka: najlepiej rozpoznawana klasa (93-99% accuracy)
- Background: minimalne false positives u modeli pretrained

F1-Score per klasa (modele pretrained):

- Concrete_bench: F1 ≈ 0.95 -0.98 (najlepiej)
- Black_lounger: F1 ≈ 0.92 -0.96
- Red_lounger: F1 ≈ 0.90 -0.95
- Gray_bin: F1 ≈ 0.85 -0.92 (najtrudniejsza)

Precision vs Recall: Modele pretrained osiągają wysoką precyzję (>98%) przy dobrej czułości (>96%), co oznacza mało false positives przy zachowaniu dobrej detekcji obiektów.

Model Comparison: Confusion Matrix Normalized



4.2 Wydajność w czasie rzeczywistym

Po optymalizacji interfejsu wszystkie modele osiągnęły wydajność pozwalającą na przetwarzanie wideo w czasie rzeczywistym przy około 30 FPS. Model YOLOv5n działał najszybciej z minimalnymi opóźnieniami, YOLOv5s oferował dobrą równowagę między szybkością a jakością, podczas gdy YOLOv5m był najwolniejszy, ale nadal pozwalał na płynne przetwarzanie obrazu.

4.3 Wydajność na filmie testowym vs metryki walidacyjne

Kluczowy problem - rozbieżność mAP vs praktyczne użycie:

Pomimo że modele pretrained osiągają bardzo wysokie wyniki walidacyjne (96.4-97.1% [mAP@0.5](#)), w praktyce wszystkie wykazują znaczące problemy z klasyfikacją. Najlepsze modele YOLOv5n_pretrained i YOLOv5s_pretrained zapewniają najbardziej stabilną detekcję obiektów, ale nie są wolne od błędów klasyfikacyjnych.

Wszystkie modele pretrained systematycznie mylą kolory leżaków - czerwone klasyfikowane są jako czarne i odwrotnie. Podobnie problematyczna jest konfuzja między ławkami a śmietnikami ze względu na ich podobny prostokątny kształt. Co więcej, te błędy klasyfikacyjne są niestabilne - ten sam obiekt może być klasyfikowany na przemian jako różne klasy w kolejnych klatkach filmu.

YOLOv5m mimo najwyższych metryk (97.1% mAP) generuje dodatkowy problem w postaci częstych podwójnych detekcji, gdzie jeden obiekt jest oznaczany dwoma nakładającymi się bounding boxami z różnymi etykietami. Ten efekt znacznie pogarsza użyteczność praktyczną modelu.

4.4 Pretrained vs From Scratch - dramatyczna różnica

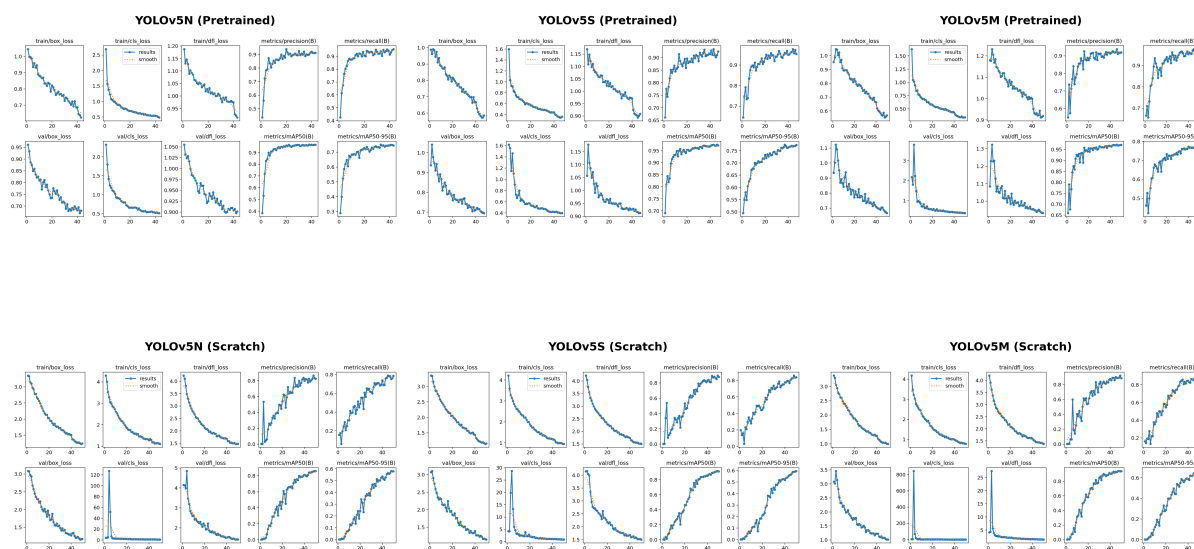
Modele pretrained, mimo wspomnianych problemów klasyfikacyjnych, zapewniają względnie stabilne wykrywanie obiektów przez cały film. Ich główne trudności dotyczą klasyfikacji, a nie lokalizacji obiektów, a bounding boxy pozostają stosunkowo stabilne między klatkami.

Modele from scratch okazały się znacznie gorsze w praktycznym użyciu. YOLOv5n_scratch praktycznie nie wykrywa żadnych obiektów mimo osiągnięcia 85.7% mAP w walidacji. YOLOv5s_scratch generuje sporadyczne detekcje z wysokim poziomem false positives. Najgorszy jest YOLOv5m_scratch, który wykrywa obiekty praktycznie wszędzie, tworząc niestabilne i chaotyczne detekcje, gdzie bounding boxy skaczą nieprzewidywalnie między klatkami.

Kluczowy paradoks metryk vs rzeczywistość: Metryki walidacyjne sugerowały użyteczność modeli from scratch (85-93% mAP), podczas gdy w praktyce są one praktycznie bezużyteczne. Potwierdza to znaczący overfitting do konkretnych

fragmentów zbioru danych i podkreśla konieczność testowania modeli w rzeczywistych scenariuszach użycia, a nie tylko na podstawie metryk walidacyjnych.

Model Comparison: Results



5. Wnioski

5.1 Kluczowe obserwacje

1. Domain gap ma krytyczne znaczenie - różnica między zdjęciami a video wymagała augmentacji
2. Metryki treningowe mogą mylić - wzorowe loss curves nie gwarantują dobrej detekcji
3. Pretrained modele wyraźnie lepsze od trenowania from scratch na małym dataset'ie
4. CPU training możliwy ale bardzo czasochłonny dla większych modeli

5.2 Praktyczne rekomendacje

- Zawsze testować modele na docelowych danych, nie tylko na metrykach
- Augmentacja danymi z target domain jest kluczowa
- Dla CPU lepiej ograniczyć się do YOLOv5n/s
- Pretrained weights znacznie przyspieszają konwergencję

5.3 Dalsze kierunki

- Test na większej liczbie filmów testowych
- Eksperyment z różnymi strategiami augmentacji
- Porównanie z nowszymi wersjami YOLO (v8, v11)
- Ewentualne przeniesienie na GPU dla większych modeli

Raport przygotowany w ramach projektu 5 z rozpoznawania obiektów. Dane zebrane i oznaczone samodzielnie, wszystkie eksperymenty przeprowadzone na własnym sprzęcie.